

Scientific Computing

NumPy

NumPy (Numerical Python) is the foundational open-source library for scientific computing in Python. It provides high-performance tools for working with large, multi-dimensional arrays and matrices, along with a vast collection of high-level mathematical functions.

Key Features

- **The ndarray Object:** A powerful N-dimensional array object that is the core of the library. Unlike standard Python lists, these arrays are stored in contiguous memory, making them up to 50x faster for numerical operations.
- **Vectorization & Broadcasting:** These allow you to perform element-wise operations on entire arrays without writing explicit loops, significantly simplifying code and increasing speed.
- **Mathematical Toolset:** Includes comprehensive routines for linear algebra, Fourier transforms, random number generation, and basic statistics.
- **Ecosystem Foundation:** NumPy is a "cornerstone" for other major data science libraries like Pandas, SciPy, and scikit-learn.

To use NumPy, you typically install it via a package manager and import it using the standard `np` alias:

```
import numpy as np
# Create a 2D array
arr = np.array([[1, 2, 3], [4, 5, 6]])
# Perform a vectorized operation
result = arr * 2
```

Arrays in Numpy

A NumPy array is a structured collection of elements of the same data type stored in a table format.

- The number of dimensions is called the rank and the size along each dimension is called the shape.
- In NumPy, arrays are called ndarray and elements are accessed using square brackets `[]`, often created from nested Python lists.

Creating a Numpy Array

Arrays in Numpy can be created by multiple ways, with various number of Ranks, defining the size of the Array. Arrays can also be created with the use of various data types such as lists, tuples, etc. The type of the resultant array is deduced from the type of the elements in the sequences.

***Note:** Type of array can be explicitly defined while creating the array.*

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr)
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr)
arr = np.array((1, 3, 2))
print(arr)
```

Output

```
[1 2 3]
[[1 2 3]
 [4 5 6]]
[1 3 2]
```

Accessing the Array Index

In a numpy array, indexing or accessing the array index can be done in multiple ways. To print a range of an array, slicing is done. Slicing of an array is defining a range in a new array which is used to print a range of elements from the original array. Since, sliced array holds a range of elements of the original array, modifying content with the help of sliced array modifies the original array content.

```
import numpy as np
arr = np.array([[-1, 2, 0, 4], [4, -0.5, 6, 0], [2.6, 0, 7, 8], [3, -7, 4, 2.0]])
arr2 = arr[:2, ::2]
print ("first 2 rows and alternate columns(0 and 2):\n", arr2)
arr3 = arr[[1, 1, 0, 3], [3, 2, 1, 0]]
print ("\nElements at indices (1, 3), (1, 2), (0, 1), (3, 0):\n", arr3)
```

Output

first 2 rows and alternate columns(0 and 2):

```
[[ -1.  0.]
```

```
 [ 4.  6.]]
```

Elements at indices (1, 3), (1, 2), (0, 1), (3, 0):

```
[0. 6. 2. 3.]
```

Basic Array Operations

In numpy, arrays allow a wide range of operations which can be performed on a particular array or a combination of Arrays. These operations include some basic Mathematical operation as well as Unary and Binary operations.

```
import numpy as np
```

```
a = np.array([[1, 2], [3, 4]])
```

```
b = np.array([[4, 3], [2, 1]])
```

```
print ("Adding 1 to every element:", a + 1)
```

```
print ("\nSubtracting 2 from each element:", b - 2)
```

```
print ("\nSum of all array elements: ", a.sum())
```

```
print ("\nArray sum:\n", a + b)
```

Output

```
Adding 1 to every element: [[2 3]
```

```
 [4 5]]
```

```
Subtracting 2 from each element: [[ 2  1]
```

```
 [ 0 -1]]
```

```
Sum of all array elements: 10
```

```
Array sum:
```

```
[[5 5]
```

```
 [5 5]]
```

Data Types in Numpy

A NumPy array is a table of elements (usually numbers) of the same data type, indexed by a tuple of positive integers. Each array has a dtype that defines the type of its elements and how they are stored in memory. NumPy provides many numeric data types and you can either let NumPy guess the type or specify it explicitly when creating an array.

Constructing a Datatype Object

In Numpy, datatypes of Arrays need not to be defined unless a specific datatype is required. Numpy tries to guess the datatype for Arrays which are not predefined in the constructor function.

```
import numpy as np
x = np.array([1, 2])
print(x.dtype)
x = np.array([1.0, 2.0])
print(x.dtype)
x = np.array([1, 2], dtype = np.int64)
print(x.dtype)
```

Output

```
int64
float64
int64
```

Math Operations on Data Type array

In Numpy arrays, basic mathematical operations are performed element-wise on the array. These operations are applied both as operator overloads and as functions. Many useful functions are provided in Numpy for performing computations on Arrays such as sum for addition of Array elements, T for Transpose of elements, etc.

Script	Output
<pre>import numpy as np arr1 = np.array([[4, 7], [2, 6]], dtype = np.float64) arr2 = np.array([[3, 6], [2, 8]], dtype = np.float64)</pre>	
<pre>Sum = np.add(arr1, arr2) print(Sum)</pre>	<pre>[[7. 13.] [4. 14.]]</pre>
<pre>Sum1 = np.sum(arr1) print(Sum1)</pre>	<pre>19.0</pre>
<pre>Sqrt = np.sqrt(arr1) print(Sqrt)</pre>	<pre>[[2. 2.64575131] [1.41421356 2.44948974]]</pre>
<pre>Trans_arr = arr1.T</pre>	<pre>[[4. 2.]</pre>

```
print(Trans_arr)
```

```
[7. 6.]
```

Pandas (DataFrames)

Pandas is a powerful, open-source Python library essential for data manipulation and analysis. It provides intuitive data structures and functions designed to make working with structured (tabular, multidimensional, time-series) data fast, easy, and expressive.

A Pandas DataFrame is a two-dimensional table-like structure in Python where data is arranged in rows and columns. It's one of the most commonly used tools for handling data and makes it easy to organize, analyze and manipulate data. It can store different types of data such as numbers, text and dates across its columns. The main parts of a DataFrame are:

- Data: Actual values in the table.
- Rows: Labels that identify each row.
- Columns: Labels that define each data category.

No	Name	Department	Year	Age
1	Hailelassie Zeferu Abel	Software Engineering	4	24
2	Worku Abenezer Girma	Computer Science	4	25
3	Abenezer Reta Tamirat	Computer Science	4	25
4	Abraham Melisew Fireda	Information Systems	4	26
5	Hailu Abraham Niguse	Computer Science	4	25
6	Weldetinsae Adonay Gebrehiwot	Information Systems	3	24
7	Debebe Gebremeskel Akin	Software Engineering	3	26
8	Teferi Amanuel Abebaw	Computer Science	3	25
9	Muche Amanuel Kahsu	Information Systems	3	25

As per the above table, Name, Department, Year and Age are different columns (attribute) consisting of Nine rows with different data in the table.

Creating a Pandas DataFrame

Pandas allows us to create a DataFrame from many data sources. We can create DataFrames directly from Python objects like lists and dictionaries or by reading data from external files like CSV, Excel or SQL databases. Here are some ways by which we create a dataframe:

1. Creating DataFrame using a List

If we have a simple list of data, we can easily create a DataFrame by passing that list to the `pd.DataFrame()` function.

```
import pandas as pd
lst = ['Software Engineering', 'Computer Science',
       'Information Technology', 'Data Science', 'Computer
Engineering']
df = pd.DataFrame(lst)
print(df)
```

	0
0	Software Engineering
1	Computer Science
2	Information Technology
3	Data Science
4	Computer Engineering

2. Creating DataFrame from dict of ndarray/lists

We can create a DataFrame from a dictionary where the keys are column names and the values are lists or arrays.

- All arrays/lists must have the same length.
- If an index is provided, it must match the length of the arrays.
- If no index is provided, Pandas will use a default range index (0, 1, 2, ...).

```
import pandas as pd

data = {'Name':['Michael', 'Eyob', 'Bethelhem', 'Martha'],
        'Department':['Computer Science', 'Information
Technology', 'Information Systems', 'Computer Engineering'],
        'Age':[20, 21, 19, 18]}
df = pd.DataFrame(data)
print(df)
```

	Name	Department	Age
0	Michael	Computer Science	20
1	Eyob	Information Technology	21

2	Bethelhem	Information Systems	19
3	Martha	Computer Engineering	18

Rows and Columns in Pandas DataFrame

We can perform basic operations on rows/columns like selecting, deleting, adding and renaming.

1. Column Selection

In Order to select a column in Pandas DataFrame, we can either access the columns by calling them by their columns name.

```
import pandas as pd

data = {'Name': ['Eyob', 'Michael', 'Bethelhem', 'Martha'],
        'Age': [27, 24, 22, 32],
        'Address': ['Addis Ababa', 'Mekelle', 'Bahirdar', 'Shashemene'],
        'Qualification': ['MSc', 'MA', 'MCA', 'PhD']}

df = pd.DataFrame(data)
print(df[['Name', 'Qualification']])
```

Output:

	Name	Qualification
0	Eyob	MSc
1	Michael	MA
2	Bethelhem	MCA
3	Martha	PhD

2. Row Selection

Pandas provide unique methods for selecting rows from a Data frame. DataFrame.loc[] method is used for label-based selection. Here we'll be using nba.csv dataset in below examples for better understanding.

```
import pandas as pd
#assuming the file nba.csv is in the same directory with
script
data = pd.read_csv("nba.csv", index_col="Name")
first = data.loc["Avery Bradley"]
second = data.loc["R.J. Hunter"]
print(first, "\n\n", second)
```

Output:

Team	Boston Celtics
------	----------------

```

Number          0.0
Position        PG
Age            25.0
Height         6-2
Weight        180.0
College        Texas
Salary       7730337.0
Name: Avery Bradley, dtype: object

Team      Boston Celtics
Number    28.0
Position  SG
Age       22.0
Height    6-5
Weight    185.0
College   Georgia State
Salary    1148640.0
Name: R.J. Hunter, dtype: object

```

Indexing and Selecting Data in Pandas DataFrame

Indexing in pandas means simply selecting particular rows and columns of data from a DataFrame. It allows us to access subsets of data such as:

- Selecting all rows and some columns.
- Selecting some rows and all columns.
- Selecting a specific subset of rows and columns.

Indexing can also be known as **Subset Selection**.

1. Indexing a Dataframe using indexing operator []

The indexing operator [] is the basic way to select data in Pandas. We can use this operator to access columns from a DataFrame. This method allows us to retrieve one or more columns. The .loc and .iloc indexers also use the indexing operator to make selections. In order to select a single column, we simply put the name of the column in-between the brackets.

```

import pandas as pd
data = pd.read_csv("nba.csv", index_col = "Name")
first = data["Age"]
print(first)

```

Output:

```

Name
Avery Bradley    25.0

```



```
Jae Crowder      25.0
John Holland     27.0
R.J. Hunter      22.0
Jonas Jerebko    29.0
...
Shelvin Mack     26.0
Raul Neto        24.0
Tibor Pleiss     26.0
Jeff Withey      26.0
NaN              NaN
Name: Age, Length: 458, dtype: float64
```

2. Indexing a DataFrame using .loc[]

The .loc method is used to select data by label. This means it uses the row and column labels to access specific data points. .loc[] is versatile because it can select both rows and columns simultaneously based on labels. In order to select a single row using .loc[], we put a single row label in a .loc function.

```
import pandas as pd
data = pd.read_csv("nba.csv", index_col = "Name")
first = data.loc["Avery Bradley"]
second = data.loc["R.J. Hunter"]
print(first, "\n\n", second)
```

Output:

```
Team      Boston Celtics
Number      0.0
Position      PG
Age      25.0
Height      6-2
Weight      180.0
College      Texas
Salary      7730337.0
Name: Avery Bradley, dtype: object
```

```
Team      Boston Celtics
Number      28.0
Position      SG
Age      22.0
Height      6-5
Weight      185.0
College      Georgia State
Salary      1148640.0
Name: R.J. Hunter, dtype: object
```

3. Indexing a DataFrame using .iloc[]

The .iloc() method allows us to select data based on integer position. Unlike .loc[] (which uses labels) .iloc[] requires us to specify row and column positions as integers

(0-based indexing). In order to select a single row using `.iloc[]`, we can pass a single integer to `.iloc[]` function.

```
import pandas as pd
data = pd.read_csv("nba.csv", index_col = "Name")
result = data.iloc[3]
print(result)
```

Output:

```
Team      Boston Celtics
Number      28.0
Position      SG
Age      22.0
Height      6-5
Weight      185.0
College      Georgia State
Salary      1148640.0
Name: R.J. Hunter, dtype: object
```

Working with Missing Data

Missing Data can occur when no information is available for one or more items or for an entire row/column. In Pandas missing data is represented as NaN (Not a Number). Missing data can be problematic in real-world datasets where data is incomplete. Pandas provides several methods to handle such missing data effectively:

1. Checking for Missing Values using `isnull()` and `notnull()`

To check for missing values (NaN) we can use two useful functions:

`isnull()`: It returns True for NaN (missing) values and False otherwise.

`notnull()`: It returns the opposite, True for non-missing values and False for NaN values.

```
import pandas as pd
import numpy as np
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, 45, 56, np.nan],
        'Third Score':[np.nan, 40, 80, 98]}
df = pd.DataFrame(dict)
print(df.isnull())
```

Output:

```
First Score  Second Score  Third Score
```

0	False	False	True
1	False	False	False
2	True	False	False
3	False	True	False

2. Filling Missing Values using fillna(), replace() and interpolate()

In order to fill null values in a dataset, we use fillna(), replace() and interpolate function these functions replace NaN values with some value of their own. All these function help in filling a null values in datasets of a DataFrame. Interpolate() function is used to fill NA values in the dataframe but it uses various interpolation technique to fill the missing values rather than hard-coding the value.

```
import pandas as pd
import numpy as np
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, 45, 56, np.nan],
        'Third Score': [np.nan, 40, 80, 98]}
df = pd.DataFrame(dict)
print(df.fillna(0))
```

	First Score	Second Score	Third Score
0	100.0	30.0	0.0
1	90.0	45.0	40.0
2	0.0	56.0	80.0
3	95.0	0.0	98.0

3. Dropping Missing Values using dropna()

If we want to remove rows or columns with missing data we can use the dropna() method. This method is flexible which allows us to drop rows or columns depending on the configuration.

```
import pandas as pd
import numpy as np
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, np.nan, 45, 56],
        'Third Score': [52, 40, 80, 98],
        'Fourth Score': [np.nan, np.nan, np.nan, 65]}
df = pd.DataFrame(dict)
print(df)
```

Output:

	First Score	Second Score	Third Score	Fourth Score
0	100.0	30.0	52	NaN
1	90.0	NaN	40	NaN
2	NaN	45.0	80	NaN
3	95.0	56.0	98	65.0

Now we drop rows with at least one Nan value (Null value).

```
import pandas as pd
import numpy as np
dict = {'First Score':[100, 90, np.nan, 95],
        'Second Score': [30, np.nan, 45, 56],
        'Third Score':[52, 40, 80, 98],
        'Fourth Score':[np.nan, np.nan, np.nan, 65]}
df = pd.DataFrame(dict)
print(df.dropna())
```

Output:

	First Score	Second Score	Third Score	Fourth Score
3	95.0	56.0	98	65.0

Iterating over rows and columns

Iteration refers to the process of accessing each item one at a time. In Pandas, it means iterating through rows or columns in a DataFrame to access or manipulate the data. We can iterate over rows and columns to extract values or perform operations on each item.

1. Iterating Over Rows

There are several ways to iterate over the rows of a Pandas DataFrame and three common methods are:

1. `iteritems()`
2. `iterrows()`
3. `itertuples()`

Each method provides different ways to iterate over the rows which depends on our specific needs.

```
import pandas as pd
dict = {'name':["Dereje", "Dagmawit", "Eyasu", "Yishak"],
        'degree': ["MBA", "BSC", "MSc", "PhD"],
        'score':[90, 40, 80, 98]}
df = pd.DataFrame(dict)
print(df)
```

Output:

	name	degree	score
0	Dereje	MBA	90
1	Dagmawit	BSC	40

2	Eyasu	MSc	80
3	Yishak	PhD	98

Here we apply **iterrows()** function in order to get a each element of rows.

```
import pandas as pd
dict = {'name':["Dereje", "Dagmawit", "Eyasu", "Yishak"],
        'degree': ["MBA", "BSC", "MSc", "PhD"],
        'score': [90, 40, 80, 98]}
df = pd.DataFrame(dict)
for i, j in df.iterrows():
    print(i, j)
    print()
```

Output:

```
0 name      Dereje
degree      MBA
score       90
Name: 0, dtype: object

1 name      Dagmawit
degree      BSC
score       40
Name: 1, dtype: object

2 name      Eyasu
degree      MSc
score       80
Name: 2, dtype: object

3 name      Yishak
degree      PhD
score       98
Name: 3, dtype: object
```

2. Iterating Over Columns

In order to iterate over columns, we need to create a list of dataframe columns and then iterating through that list to pull out the dataframe columns.

```
import pandas as pd
dict = {'name':["Dereje", "Dagmawit", "Eyasu", "Yishak"],
        'degree': ["MBA", "BSC", "MSc", "PhD"],
        'score': [90, 40, 80, 98]}
df = pd.DataFrame(dict)
print(df)
```

Output:

	name	degree	score
0	Dereje	MBA	90
1	Dagmawit	BSC	40

2	Eyasu	MSc	80
3	Yishak	PhD	98

Now here we iterate through columns in order to iterate through columns we first create a list of dataframe columns and then iterate through list.

```
columns = list(df)
for i in columns:
    print (df[i][2])
```

Output:

```
Eyasu
MSc
80
```

Data Cleaning & Wrangling.

Common Challenges in Raw Data

Before diving into the techniques for cleaning and preparing data, it is important to understand some of the common challenges you might encounter in raw datasets:

- **Missing Data** : One of the most common problems where values are missing, either due to data entry errors or incomplete data collection.
- **Inconsistent Data** : Another common problem due to variations in how data is recorded, such as different date formats or inconsistent use of units.
- **Duplicate Records** : In some cases, your dataset may have multiple entries that represent the same observation, which can skew analysis results.
- **Outliers** : Extreme values that can distort statistical analyses if not identified and handled appropriately.
- **Data Entry Errors** : Mistakes made during data collection, such as typographical errors or incorrect data coding.

Systematic Approach

Approaching data cleaning and wrangling systematically ensures that you don't overlook any critical steps. A structured process typically involves:

- Initial Assessment : Begin by examining the dataset to understand its structure and identify potential issues. This might include looking at the data types, ranges of values, and the presence of missing or inconsistent data.
- Cleaning the Data : Address identified issues by filling in missing values, correcting errors, and standardizing formats. Techniques vary depending on the nature of the data and the tools available.
- Transforming the Data : Once cleaned, the data may need to be transformed to fit the specific requirements of your analysis. This could involve normalizing values, aggregating data, or creating new calculated fields.
- Validating the Data : After cleaning and transforming the data, it is important to validate your work by checking for remaining issues and ensuring the dataset is ready for analysis.

Tools and Techniques

Data cleaning and preparation can be done using a variety of tools, ranging from beginner-friendly spreadsheet applications (e.g., Google Sheets and Microsoft Excel) to more advanced programming languages (e.g., Python and R). The choice of tools often depends on the complexity of the data and your familiarity and preferences with these tools.

- Spreadsheet Applications : Tools like Google Sheets and Microsoft Excel are ideal for beginners, as they offer intuitive interfaces for basic data cleaning tasks such as sorting, filtering, removing duplicates, and performing simple calculations. They are also very accessible and widely used, making them a great starting point for those new to data work.
- Python : A powerful programming language that offers libraries like pandas and NumPy, which are specifically designed for data manipulation. Python is highly flexible and allows for more complex data cleaning tasks.
- R : Another programming language popular in data science, with a rich set of packages like tidyverse that are tailored for data cleaning and transformation. R is particularly strong in statistical analysis and data visualization.

Data Cleaning Techniques

Handling Missing Data

Missing data is a common issue in datasets and can arise from various sources, such as incomplete data collection or data entry errors. How you handle missing data depends on the nature of your analysis and the extent of the missing values. Below are some common methods for handling missing data:

1. **Removal:** In some cases, it might be appropriate to remove rows or columns with missing data, especially if the missingness is random and minimal.
2. **Imputation:** Replacing missing values with substituted data, such as the mean, median, or a predicted value based on other variables.
3. **Flagging:** Adding a flag to indicate that data is missing, which can be useful in tracking how missing data might affect your analysis.

Let's use this dataset as an example:

ID	Name	Age	Salary
Eth - 101	Hailelassie Zeferu Abel	25	55000
Eth - 102	Worku Abenezer Girma		48000
Eth - 103	Abenezer Reta Tamirat	29	
Eth - 104	Abraham Melisew Fireda	33	62000
Eth - 105	Hailu Abraham Niguse		

Method 1: Removal

```
import pandas as pd

# Create the dataset
data = {'ID': ['Eth - 101', 'Eth - 102', 'Eth - 103', 'Eth - 104', 'Eth - 105'],
        'Name': ['Hailelassie Zeferu Abel', 'Worku Abenezer Girma', 'Abenezer Reta Tamirat', 'Abraham Melisew Fireda', 'Hailu Abraham Niguse'],
        'Age': [25, None, 29, 33, None],
        'Salary': [55000, 48000, None, 62000, None]}

df = pd.DataFrame(data)

# Remove rows with missing values
df_cleaned = df.dropna()

print(df_cleaned)
```


ID	Name	Age	Salary
Eth - 101	Hailelassie Zeferu Abel	25	55000
Eth - 104	Abraham Melisew Fireda	33	62000

Method 2: Imputation

```
# Impute missing values with the mean of the column
for column in df.columns:
    if df[column].dtype == 'float64' or df[column].dtype == 'int64':
        df[column].fillna(df[column].mean(), inplace=True)
print(df)
```

ID	Name	Age	Salary
Eth - 101	Hailelassie Zeferu Abel	25	55000
Eth - 102	Worku Abenezer Girma	29	48000
Eth - 103	Abenezer Reta Tamirat	29	55000
Eth - 104	Abraham Melisew Fireda	33	62000
Eth - 105	Hailu Abraham Niguse	29	55000

Method 3: Flagging

Depending on the nature of your data and the goal of your analysis, you might want to consider different flagging strategies. If your dataset is small or if individual column values are critical to your analysis, column-based flagging is preferable as it gives you a detailed view. But if your dataset is large or you are primarily concerned with overall row completeness, row-based flagging might be simpler and more effective. You can also use both strategies, first flagging columns for missing values and then flagging rows that contain any missing values, allowing for both granular and holistic views of missing data.

```
import pandas as pd
# Create a column-based flag
for column in df.columns:
    if df[column].isnull().any():
        df[f'{column}_Flag'] = df[column].isna().apply(lambda x: 'Missing'
if x else 'Not Missing')
# Create a row-based flag
df['Missing_Flag'] = df.isnull().any(axis=1).apply(lambda x: 'Missing' if x
else 'Not Missing')
```

```
print(df)
```

ID	Name	Age	Salary	ID_Flagged	Name_Flagged	Age_Flagged	Salary_Flagged	Missing_Flagged
Eth - 101	Hailelassie Zeferu Abel	25	55000	Not Missing	Not Missing	Not Missing	Not Missing	Not Missing
Eth - 102	Worku Abenezer Girma		48000	Not Missing	Not Missing	Missing	Not Missing	Missing
Eth - 103	Abenezer Reta Tamirat	29		Not Missing	Not Missing	Not Missing	Missing	Missing
Eth - 104	Abraham Melisew Fireda	33	62000	Not Missing	Not Missing	Not Missing	Not Missing	Not Missing
Eth - 105	Hailu Abraham Niguse			Not Missing	Not Missing	Missing	Missing	Missing

Removing Duplicates

Duplicates can skew your analysis by giving undue weight to certain data points. Identifying and removing duplicates is crucial to ensure the accuracy of your results. Below are some common methods for identifying duplicates:

1. Exact Match: Finding rows that are completely identical across all columns.
2. Partial Match: Identifying duplicates based on a subset of columns (e.g., the same name and date, but different addresses).

ID	Name	Age	Salary	City
Eth - 101	Hailelassie Zeferu Abel	25	55000	Addis Ababa
Eth - 102	Worku Abenezer Girma	29	48000	Adama
Eth - 103	Abenezer Reta Tamirat	29	55000	Addis Ababa
Eth - 104	Abraham Melisew Fireda	33	62000	Adama
Eth - 105	Hailu Abraham Niguse	29	55000	Addis Ababa
Eth - 102	Worku Abenezer Girma	29	48000	Adama
Eth - 104	Abraham Melisew Fireda	33	62000	Adama
Eth - 106	Hailu Abraham Niguse	29	55000	Addis Ababa

Let's use this dataset as an example:

In this dataset, you can see that the column with ID Eth – 102 and Eth – 104 each appear twice, which are considered duplicates. However, there are two presumably distinct observations that share the name "**Hailu Abraham Niguse**" but have different IDs, ages, salaries, and cities. The approach taken to remove duplicates could significantly impact the final results.

```

import pandas as pd

# Sample dataset
data = {'ID': ['Eth - 101', 'Eth - 102', 'Eth - 103', 'Eth - 104', 'Eth - 105', 'Eth - 102', 'Eth - 104', 'Eth - 106'],
        'Name': ['Hailesllassie Zeferu Abel', 'Worku Abenezer Girma', 'Abenezer Reta Tamirat', 'Abraham Melisew Fireda', 'Hailu Abraham Niguse', 'Worku Abenezer Girma', 'Abraham Melisew Fireda', 'Hailu Abraham Niguse'],
        'Age': [25, 29, 29, 33, 29, 29, 33, 29],
        'Salary': [55000, 48000, 55000, 62000, 55000, 48000, 62000, 55000],
        'City': ['Addis Ababa', 'Adama', 'Addis Ababa', 'Adama', 'Addis Ababa', 'Adama', 'Adama', 'Addis Ababa']}

# Original Dataset
df = pd.DataFrame(data)
print("Original Dataset")
print(df, '\n')

```

ID	Name	Age	Salary	City
Eth - 101	Hailesllassie Zeferu Abel	25	55000	Addis Ababa
Eth - 102	Worku Abenezer Girma	29	48000	Adama
Eth - 103	Abenezer Reta Tamirat	29	55000	Addis Ababa
Eth - 104	Abraham Melisew Fireda	33	62000	Adama
Eth - 105	Hailu Abraham Niguse	29	55000	Addis Ababa
Eth - 102	Worku Abenezer Girma	29	48000	Adama
Eth - 104	Abraham Melisew Fireda	33	62000	Adama
Eth - 106	Hailu Abraham Niguse	29	55000	Addis Ababa

```

# Exact Match – based on all columns
df_cleaned = df.drop_duplicates()
print("Duplicates Removed – Exact Match")
print(df_cleaned, '\n')

```

Duplicates Removed – Exact Match

ID	Name	Age	Salary	City
Eth - 101	Hailesllassie Zeferu Abel	25	55000	Addis Ababa
Eth - 102	Worku Abenezer Girma	29	48000	Adama

Eth - 103	Abenezer Reta Tamirat	29	55000	Addis Ababa
Eth - 104	Abraham Melisew Fireda	33	62000	Adama
Eth - 105	Hailu Abraham Niguse	29	55000	Addis Ababa
Eth - 106	Hailu Abraham Niguse	29	55000	Addis Ababa

```
# Partial Match – based on specific columns only (e.g., 'ID')
df_cleaned_id = df.drop_duplicates(subset=['ID'])
print("Duplicates Removed – Partial Match based on ID")
print(df_cleaned_id, '\n')
```

Duplicates Removed – Partial Match based on ID

ID	Name	Age	Salary	City
Eth - 101	Hailelassie Zeferu Abel	25	55000	Addis Ababa
Eth - 102	Worku Abenezer Girma	29	48000	Adama
Eth - 103	Abenezer Reta Tamirat	29	55000	Addis Ababa
Eth - 104	Abraham Melisew Fireda	33	62000	Adama
Eth - 105	Hailu Abraham Niguse	29	55000	Addis Ababa
Eth - 106	Hailu Abraham Niguse	29	55000	Addis Ababa

```
# Partial Match – based on specific columns only (e.g., 'ID')
df_cleaned_name = df.drop_duplicates(subset=['Name'])
print("Duplicates Removed – Partial Match based on Name")
print(df_cleaned_name, '\n')
```

Duplicates Removed - Partial Match based on Name

ID	Name	Age	Salary	City
Eth - 101	Hailelassie Zeferu Abel	25	55000	Addis Ababa
Eth - 102	Worku Abenezer Girma	29	48000	Adama
Eth - 103	Abenezer Reta Tamirat	29	55000	Addis Ababa
Eth - 104	Abraham Melisew Fireda	33	62000	Adama
Eth - 105	Hailu Abraham Niguse	29	55000	Addis Ababa

```
# Partial Match – based on specific columns only (e.g., 'ID' and 'Name')
```

```
df_cleaned_specific = df.drop_duplicates(subset=['ID', 'Name'])
print("Duplicates Removed - Partial Match based on ID and Name")
print(df_cleaned_specific, '\n')
```

Duplicates Removed - Partial Match based on ID and Name

ID	Name	Age	Salary	City
Eth - 101	Hailelassie Zeferu Abel	25	55000	Addis Ababa
Eth - 102	Worku Abenezer Girma	29	48000	Adama
Eth - 103	Abenezer Reta Tamirat	29	55000	Addis Ababa
Eth - 104	Abraham Melisew Fireda	33	62000	Adama
Eth - 105	Hailu Abraham Niguse	29	55000	Addis Ababa
Eth - 106	Hailu Abraham Niguse	29	55000	Addis Ababa

Normalizing Data Formats

Data collected from various sources might be in different formats, making it difficult to analyze. Normalizing these formats ensures consistency across your dataset. Below are some common tasks in normalization:

- **Standardizing Dates:** Converting dates to a consistent format (e.g., YYYY-MM-DD).
- **Text Case Standardization:** Ensuring that text data is consistently in uppercase or lowercase.
- **Unit Conversion:** Converting measurements to a common unit (e.g., converting all weights to kilograms).

Standardizing Dates

Event ID	Event Date	Product Name	Weight
E-001	01/12/2023	apple	70 kg
E-002	December 2, 2023	Banana	100 lbs
E-003	2023-12-03	ORANGE	88 KG
E-004	12.04.2023	BaNAAna	180 pound
E-005	5th Dec 2023	Apple	94 kilograms

```
import pandas as pd
from dateutil.parser import parse
# Sample data
```

```

data = {
    'Event ID': ['E-001', 'E-002', 'E-003', 'E-004', 'E-005'],
    'Event Date': ['01/12/2023', 'December 2, 2023', '2023-12-03', '12.04.2023', '5th Dec 2023'],
    'Product Name': ['apple', 'Banana', 'ORANGE', 'BaNAAna', 'Apple'],
    'Weight' : ['70 kg', '100 lbs', '88 KG', '180 pound', '94 kilograms']
}

df = pd.DataFrame(data)
print("Original Dataset")
print(df, '\n')

```

```

# Custom function to parse dates
def parse_date(date_str):
    try:
        return parse(date_str, dayfirst=False)
    except ValueError:
        return pd.NaT

# Apply the custom function
df['Event Date'] = df['Event Date'].apply(parse_date)

# Standardize format to YYYY-MM-DD
df['Event Date'] = df['Event Date'].dt.strftime('%Y-%m-%d')

print("Result")
print(df)

```

Event ID	Event Date	Product Name	Weight
E-001	2023-01-12	apple	70 kg
E-002	2023-12-02	banana	100 lbs
E-003	2023-12-03	orange	88 KG
E-004	2023-12-04	banana	180 pound
E-005	2023-12-05	apple	94 kilograms

Text Case Standardization

```

df = pd.DataFrame(data)

# Convert 'Product Name' to lowercase
df['Product Name'] = df['Product Name'].str.lower()

```

```
print(df)
```

Event ID	Event Date	Product Name	Weight	Weight in kg
E-001	2023-01-12	apple	70 kg	70.00000
E-002	2023-12-02	banana	100 lbs	45.35920
E-003	2023-12-03	orange	88 KG	88.00000
E-004	2023-12-04	banana	180 pound	81.64656
E-005	2023-12-05	apple	94 kilograms	94.00000

Unit Conversion

```
df = pd.DataFrame(data)
# Extract numeric value and unit
df['Numeric Value'] = df['Weight'].str.extract(r'(\d+\.? \d*)').astype(float)

# Use str to convert the extracted unit to lowercase
df['Unit'] = df['Weight'].str.extract(r'([a-zA-Z]+)')[0].str.lower() # Access the first element of the returned DataFrame, which is a Series

# Conversion function
def convert_to_kg(row):
    if row['Unit'] in ['kg', 'kilograms']:
        return row['Numeric Value']
    elif row['Unit'] in ['lbs', 'pound']:
        return row['Numeric Value'] * 0.453592
    else:
        return None # or handle unknown units

df['Weight in kg'] = df.apply(convert_to_kg, axis=1)
print(df[['Event ID', 'Weight', 'Weight in kg']])
```

Event ID	Event Date	Product Name	Weight	Weight in kg
E-001	2023-01-12	apple	70 kg	70.00000
E-002	2023-12-02	Banana	100 lbs	45.35920
E-003	2023-12-03	ORANGE	88 KG	88.00000
E-004	2023-12-04	BaNAna	180 pound	81.64656
E-005	2023-12-05	Apple	94 kilograms	94.00000

Outlier Detection and Handling

Outliers are data points that deviate significantly from other observations in the dataset. They can be the result of errors or true anomalies, and handling them appropriately is essential for accurate analysis. Below are some common methods for handling outliers:

1. Visual Inspection: Using plots like box plots or scatter plots to identify outliers visually.
2. Statistical Methods: Using standard deviations, interquartile range (IQR), or z-scores to detect outliers.
3. Decision on Handling: Decide whether to keep, remove, or transform outliers based on their impact on the analysis.

No	Name	Department	Salary
1	Hailelassie Zeferu Abel	Software Engineering	80000
2	Worku Abenezer Girma	Computer Science	95000
3	Abenezer Reta Tamirat	Computer Science	115000
4	Abraham Melisew Fireda	Information Systems	75000
5	Hailu Abraham Niguse	Computer Science	110000
6	Weldetinsae Adonay Gebrehiwot	Information Systems	85000
7	Debebe Gebremeskel Akin	Software Engineering	100000
8	Teferi Amanuel Abebaw	Computer Science	199000
9	Muche Amanuel Kahsu	Information Systems	105000
10	Amanuel Degulaw Zerga	Software Engineering	90000
11	Amanuel Abate Nega	Computer Science	80000
12	Amanuel Solomon Hailu	Information Systems	95000

```
import pandas as pd
import matplotlib.pyplot as plt

data = {'Person ID': range(1, 13),

        'Full Name': ['Hailelassie Zeferu Abel', 'Worku Abenezer Girma', 'Aben
ezer Reta Tamirat', 'Abraham Melisew Fireda', 'Hailu Abraham Niguse', 'Weld
etinsae Adonay Gebrehiwot ', 'Debebe Gebremeskel Akin', 'Teferi Amanuel Abe
baw', 'Muche Amanuel Kahsu', 'Amanuel Degulaw Zerga', 'Amanuel Abate Nega'
, 'Amanuel Solomon Hailu'],

        'Income': [80000, 95000, 115000, 75000, 110000, 85000, 100000, 199000,
105000, 90000, 80000, 95000]

}

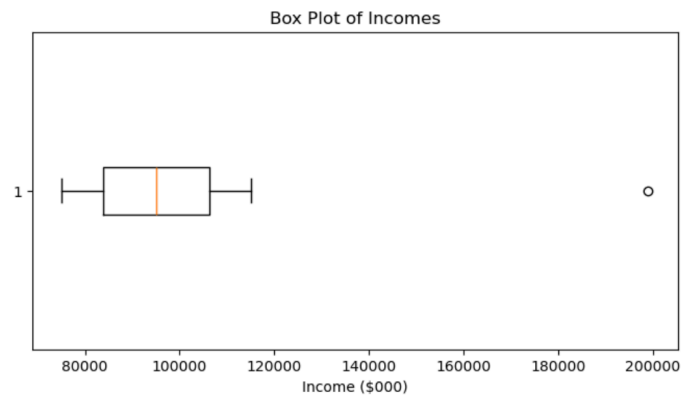
df = pd.DataFrame(data)
```

Visual Inspection

```
# Box Plot
```



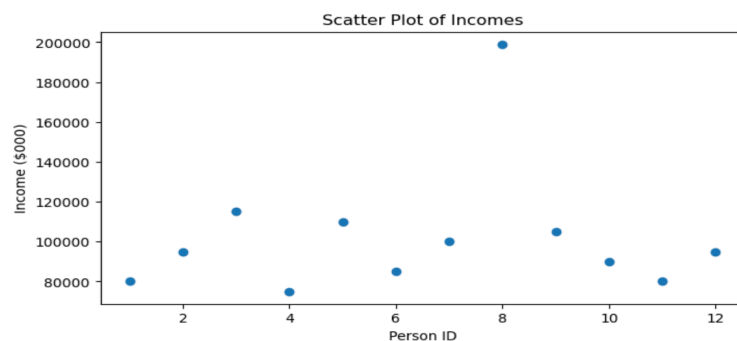
```
plt.figure(figsize=(8, 4))
plt.boxplot(df['Income'], vert=False, showfliers=True)
plt.title('Box Plot of Incomes')
plt.xlabel('Income ($000)')
plt.show()
```



Box Plot:

- The box plot shows the distribution of incomes where the income of \$199,000 appears as an outlier above the upper whisker.
-

```
# Scatter Plot
plt.figure(figsize=(8, 4))
plt.scatter(df['Person ID'], df['Income'])
plt.title('Scatter Plot of Incomes')
plt.xlabel('Person ID')
plt.ylabel('Income ($000)')
plt.show()
```



providing scattered and boxplot diagram showing the outlier

Scatter Plot: The scatter plot displays Person with ID number 8's income distinctly higher than others.

Statistical Methods

```

df = pd.DataFrame(data)
# Calculate Q1, Q3, bounds and Identify outliers
Q1 = df['Income'].quantile(0.25)
Q3 = df['Income'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df['Outlier'] = df['Income'].apply(lambda x: 'Outlier' if x > upper_bound else 'Normal')
print(df)

```

	Person	Full Name	Income	Outlier
0	1	Hailelassie Zeferu Abel	80000	Normal
1	2	Worku Abenezer Girma	95000	Normal
2	3	Abenezer Reta Tamirat	115000	Normal
3	4	Abraham Melisew Fireda	75000	Normal
4	5	Hailu Abraham Niguse	110000	Normal
5	6	Weldetinsae Adonay Gebrehiwot	85000	Normal
6	7	Debebe Gebremeskel Akin	100000	Normal
7	8	Teferi Amanuel Abebaw	199000	Outlier
8	9	Muche Amanuel Kahsu	105000	Normal
9	10	Amanuel Degulaw Zerga	90000	Normal
10	11	Amanuel Abate Nega	80000	Normal
11	12	Amanuel Solomon Hailu	95000	Normal

Decision on Handling Outliers

```

# Mean and Standard Deviation with Outlier
mean_with_outlier = df['Income'].mean()
std_with_outlier = df['Income'].std()
# Mean and Standard Deviation without Outlier
df_clean = df[df['Outlier'] == 'Normal']
mean_without_outlier = df_clean['Income'].mean()
std_without_outlier = df_clean['Income'].std()
print("Income with Outlier")
print(f"Mean: ${mean_with_outlier:,.2f}")
print(f"SD: ${std_with_outlier:,.2f}")
print("Income without Outlier")

```

```
print(f"Mean: ${mean_without_outlier:,.2f}")
print(f"SD: ${std_without_outlier:,.2f}")
```

Income with Outlier
Mean: 102,416.67
SD: 32,864.62

Income without Outlier
Mean: 93,636.36
SD: 13,055.82

Correcting Data Entry Errors

Data entry errors are a common challenge, particularly in manual data collection processes. Left unaddressed, these errors can skew analyses, lead to incorrect conclusions, and compromise the integrity of your dataset. Therefore, it is essential to identify and correct them to ensure data accuracy and reliability. Below are some common methods for detecting and fixing data entry errors:

1. Validation Rules

Validation rules help prevent errors before they even occur by setting up conditions that data must meet. This method ensures that values entered into the system are consistent, logical, and fall within expected ranges. Examples of validation rules include:

- Date of Birth cannot be a future date.
- Age should be within a reasonable range (e.g., 0 to 120 years).
- Email Address should follow the standard format (e.g., name@domain.com).

```
import pandas as pd
from datetime import datetime

# Sample data
data = {
    'Name': ['Samuel Demissie', 'Esayas Girma', 'Robel Gizachew'],
    'DOB': ['1990-01-15', '2094-05-10', '1985-07-23']
}

df = pd.DataFrame(data)

# Convert 'DOB' to datetime and flag future dates
df['DOB'] = pd.to_datetime(df['DOB'], errors='coerce')
df['Valid DOB'] = df['DOB'] <= pd.Timestamp(datetime.today())

print(df)
```

	Name	DOB	Valid DOB
0	Samuel Demissie	1990-01-15	True
1	Esayas Girma	2094-05-10	False
2	Robel Gizachew	1985-07-23	True

2. Manual Review

For key variables, especially in small datasets, a manual review of entries is often necessary. This method involves going through the dataset by hand to identify inconsistencies, missing values, or obvious mistakes. While time-consuming, manual review is highly effective in catching subtle errors that automated tools might miss, such as transposed digits (e.g., entering "1987" instead of "1897" for a birth year).

3. Automated Checks

Automated checks leverage software tools to automatically identify and flag common data entry errors. These checks can be programmed to detect outliers, incorrect formats, missing values, or other anomalies. Examples of automated checks include:

- **Range Checks:** Ensuring all values fall within predefined limits.
- **Consistency Checks:** Verifying that related fields make sense together (e.g., the “end date” of an event cannot be earlier than the “start date”).
- **Format Validation:** Check that data entries adhere to a specific format (e.g., email addresses must include "@" and a domain).
- **Spelling and Variations Detection:** Automated tools can identify spelling errors and variations in text data (e.g., “USA,” “United States,” “U.S.”). This ensures consistency in categorical data, which can otherwise lead to inaccurate groupings or analysis. Some tools can flag these inconsistencies or automatically correct them to a predefined standard.

Automated tools like Apps Script in Google Sheets, Macros in Microsoft Excel, Python, and R allow you to perform these checks efficiently, especially for larger datasets. Python and R, in particular, offer robust libraries for data validation and error correction.

Data Wrangling Techniques

Essential techniques and best practices for preparing ready-to-use data, with implementation examples in Google Sheets, Microsoft Excel, Python, and R.

Reshaping Data

Reshaping data involves changing the structure or layout of your dataset, making it easier to work with during analysis. This can include pivoting data from long to wide format (or vice versa), splitting or combining columns, and more.

Pivoting Data

Rearranging the columns and rows of your data can help you view it from different perspectives.

Patient ID	Age Group	Treatment	Response
P1	18-30	Drug A	Positive
P2	31-50	Drug B	Negative
P3	51-65	Drug A	Positive

P4	18-30	Drug C	Negative
P5	31-50	Drug B	Positive
P6	51-65	Drug C	Positive

For example, consider this clinical trials data from a study testing the effectiveness of three different drugs (Drug A, Drug B, and Drug C) on patients with a specific medical condition. The dataset records information about patients' responses to treatment across various age groups, treatment groups, and outcomes. The data is in a "long" format, where each row represents a single patient's response to a treatment.

Pivoting the data allows you to summarize patient responses to different treatments based on age group and treatment type in a "wide" format. By pivoting, you can easily compare the effectiveness of each drug across age groups and identify patterns in patient outcomes.

```
# Load the pandas package and create the dataset
import pandas as pd

data = {
    'Patient_ID': ['P1', 'P2', 'P3', 'P4', 'P5', 'P6'],
    'Age Group': ['18-30', '31-50', '51-65', '18-30', '31-50', '51-65'],
    'Treatment': ['Drug A', 'Drug B', 'Drug A', 'Drug C', 'Drug B', 'Drug C'],
    'Response': ['Positive', 'Negative', 'Positive', 'Negative', 'Positive', 'Positive']
}

df = pd.DataFrame(data)

df
```

and creating a pivoting data with the following script

```
# Create a new column with 1 to indicate occurrences
df['Count'] = 1

# Pivot the data
pivoted_data = df.pivot_table(
    index='Age Group',
    columns=['Treatment', 'Response'],
    values='Count',
    aggfunc='sum',
    fill_value=0
)

# Display the final pivoted data
```

```
print(pivoted_data)
```

	Drug A	Drug B		Drug C	
Age Group	Positive	Negative	Positive	Negative	Positive
18-30	1	0	0	1	0
31-50	0	1	1	0	0
51-65	1	0	0	0	1

Splitting and Combining Columns

In some cases, splitting columns helps separate meaningful components from a single string, making it easier to analyze specific parts of the data. For example, let's consider this data with an address column in it:

```
# Load the pandas package
import pandas as pd

# Create the dataset
data = {
    'Patient ID': ['P1', 'P2', 'P3', 'P4', 'P5'],
    'Address': ['Nigeria St, Addis Ababa, Kolfe Keranio, 07, B104', 'Russia
St, Addis Ababa, Nifas Silk, 02, 1001', 'Minilik II Avenue, Addis Ababa, Ki
rkos, 02, 12', 'Haile Gebresillasie, Addis Ababa, Lideta, 04, 112', 'Guina,
Addis Ababa, Arada, 09, 621']
}
df = pd.DataFrame(data)

df
```

```
# Split the address column
df[['Street Address', 'City', 'State', 'ZIP Code']] = df['Address'].str.spl
it(',', expand=True)

# Display the result
print(df)
```

	Patient ID	Address
0	P1	Nigeria St, Addis Ababa, Kolfe Keranio, 07, B104
1	P2	Russia St, Addis Ababa, Nifas Silk, 02, 10/1
2	P3	Minilik II Avenue, Addis Ababa, Kirkos, 02, 12
3	P4	Haile Gebresillasie, Addis Ababa, Lideta, 04, 112
4	P5	Guina, Addis Ababa, Arada, 09, 621

You can split the address column into four: street address, city, state, and ZIP code. By doing so, you will be able to sort or categorize your data by Street, city, Sub-city, Woreda and House No that you won't be able to do in the previous format.

	Patient ID	Address	Street Address	City	Subcity	Woreda	House No
0	P1	Nigeria St, Addis Ababa, Kolfe Keranio, 07, B104	Nigeria St	Addis Ababa	Kolfe Keranio	7	B104
1	P2	Russia St, Addis Ababa, Nifas Silk, 02, 10/1	Russia St	Addis Ababa	Nifas Silk	2	1001
2	P3	Minilik II Avenue, Addis Ababa, Kirkos, 02, 12	Minilik II Avenue	Addis Ababa	Kirkos	2	12
3	P4	Haile Gebresillasie, Addis Ababa, Lideta, 04, 112	Haile Gebresillasie	Addis Ababa	Lideta	4	112
4	P5	Guina, Addis Ababa, Arada, 09, 621	Guina	Addis Ababa	Arada	9	621

Combining multiple column from the above table and deleting the individual address from the above dataframe.

```
# Combine the columns back into a single Address column
df['Full Address'] = df['Street Address'] + ', ' + df['City'] + ', ' + df['Subcity'] + ', ' + df['Woreda'] + ', ' + df['House No']

# Drop the split columns (optional)
df = df.drop(columns=['Street Address', 'City', 'Subcity', 'Woreda', 'House No'])

# Display the result
print(df)
```

	Patient ID	Address	Full Address
0	P1	Nigeria St, Addis Ababa, Kolfe Keranio, 07, B104	Nigeria St, Addis Ababa, Kolfe Keranio, 07, B104
1	P2	Russia St, Addis Ababa, Nifas Silk, 02, 10/1	Russia St, Addis Ababa, Nifas Silk, 02, 10/1
2	P3	Minilik II Avenue, Addis Ababa, Kirkos, 02, 12	Minilik II Avenue, Addis Ababa, Kirkos, 02, 12
3	P4	Haile Gebresillasie, Addis Ababa, Lideta, 04, 112	Haile Gebresillasie, Addis Ababa, Lideta, 04, 112
4	P5	Guina, Addis Ababa, Arada, 09, 621	Guina, Addis Ababa, Arada, 09, 621